

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №1
Варіант 9
На тему
“Основи функціонального програмування у Python”

Виконав:
Студент групи ФЕП-22
Обиход І.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати `typing.Callable` для параметризації обчислень.

Теоретичні відомості:

- Чиста функція: детермінована (однаковий вхід $\rightarrow$ однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
- Референтна прозорість: вираз можна замінити його значенням без зміни поведінки програми.
- Побічні ефекти: I/O (`print`, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від `time`, `random`, `os.environ` тощо.
- Підхід: відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Завдання (варіант 9 - Прокат автомобілів):

Вхідні дані: клас авто, кількість днів, денна ставка, статус бронювання. Потрібно обчислити базову ціну прокату, знижку за тривалість і страховий збір. Політики знижки та страхування мають передаватися як функції (`typing.Callable`); вхідні записи не змінюються, ядро не містить глобального стану та I/O.

Хід роботи

Під час виконання лабораторної роботи було створено програмне рішення для розрахунку вартості прокату автомобілів. Архітектура проєкту побудована на засадах функціонального програмування з чітким відокремленням обчислювальної логіки від побічних ефектів. Проєкт складається з чистого ядра `core.py`, оболонки побічних ефектів `app.py`, набору тестів `tests/test_core.py`, файлу даних `data/rentals.json`, а також конфігурації інструментів `pyproject.toml` і `requirements.txt` (`pytest`, `mypy`, `black`, `ruff`).

Стартовий імперативний фрагмент (файл `starter_imperative.py`) містив типові проблеми: глобальні константи `INSURANCE_RATE` та `LONG_RENTAL_DISCOUNT` усередині обчислень, виклик `print` у циклі, мутацію вхідних словників через `r["total"] = ...` та накопичувальні змінні. У ході роботи його було переписано так, щоб жодна з цих проблем не залишилась.

Завдання 1. У модулі `core.py` описано типи предметної області `Rental` і `Report` (`TypedDict`) та реалізовано чисті функції: `base_price` (кількість днів $\times$ денна ставка), `is_confirmed` (перевірка статусу), `duration_discount_rate` (ставка знижки за порогами тривалості), `insurance_rate_for` (ставка страховки за класом авто), `with_pricing` (створює новий запис із полями `base_price`, `discount`, `insurance`, `total`, не змінюючи вхідний) та `summarize` (формує звіт). Головна функція `process_rentals_pure` приймає усі параметри явно: мінімальну кількість днів, пороги знижки та ставки страхування. Вона відбирає підтверджені прокати, обчислює ціну і повертає новий словник із кількістю, сумарним доходом та новими записами.

Розрахунок виконується за формулою: $\text{base} = \text{days} \times \text{daily_rate}$; $\text{discount} = \text{base} \times \text{rate}(\text{days})$, де ставка становить 10 % від 7 днів, 15 % від 14 і 25 % від 30 днів; $\text{insurance} = (\text{base} - \text{discount}) \times \text{rate}(\text{car_class})$, де `economy` 5 %, `comfort` 8 %, `business` 12 %, `premium` 18 %; $\text{total} = \text{base} - \text{discount} + \text{insurance}$.

Завдання 2. Щоб зробити систему гнучкою, обчислення параметризовано за допомогою `typing.Callable`. Визначено типи політик `AcceptFn` (фільтр), `DiscountFn` (знижка) та `InsuranceFn` (страховий збір). Фабрика `make_processor(accept=..., apply_discount=..., insurance_fee=...)` повертає налаштований обробник, у який будь-яку політику можна підставити без зміни коду. Реалізовано фабрики політик `make_min_days_filter`, `make_duration_discount`, `make_class_insurance`, `make_per_day_insurance` та `lift_price_fn`, що підіймає просту функцію ціни до рівня політики знижки.

Завдання 3. Усі побічні ефекти винесено в `app.py`: читання JSON-файлу виконує `load_rentals`, друк звіту - `render_report`, яка лише споживає готовий результат. Поточний час не викликається в ядрі: функції `stamp_total` і `stamp_report` приймають постачальник часу `NowFn`, а `time.time` передається з оболонки, тому ядро лишається чистим і тестується з фейковим годинником.

Додаткові вправи. Реалізовано `compose` та `pipe` для побудови знижки з кількох маленьких функцій (`make_percent_off`, `make_coupon`), фабрику множників `make_multiplier`, а також функції `mark_cancelled` і `totals_by_class`, які замість мутації списку повертають нові колекції через `list comprehension`.

Надійність реалізованого підходу доведено автоматизованими тестами `pytest` (20 тестів у файлі `tests/test_core.py`). Вони перевіряють референтну прозорість, відсутність мутації вхідних даних, коректність числових результатів, замінюваність `Callable`-політик, інжекцію часу та додаткові функції. Перевірка туру у строгому режимі не виявила помилок типізації, а `black` і `ruff` підтвердили відповідність коду стандартам форматування.

Файл core.py

```
core.py                      рядки 1-61

1  """Чисте ядро предметної області «Прокат автомобілів» (варіант 9).
2
3  Усі функції в цьому модулі:
4
5  * детерміновані - однаковий вхід завжди дає однаковий вихід;
6  * не змінюють вхідні дані - завжди повертають нові об'єкти;
7  * не виконують I/O (жодного print, файлів, мережі);
8  * не використовують глобальний стан - усі параметри передаються явно
9    або інжектуються через ``typing.Callable``.
10 """
11
12 from __future__ import annotations
13
14 from collections.abc import Callable, Iterable, Mapping, Sequence
15 from typing import TypedDict, TypeVar
16
17 # -----
18 # Типи предметної області
19 # -----
20
21
22 class Rental(TypedDict, total=False):
23     """Запис про прокат автомобіля.
24
25     Поля ``base_price``, ``discount``, ``insurance`` та ``total``
26     відсутні у вхідних даних і з'являються лише у результаті обробки.
27     """
28
29     id: int
30     car_class: str # "economy" | "comfort" | "business" | "premium"
31     days: int
32     daily_rate: float
33     status: str # "confirmed" | "pending" | "cancelled"
34     base_price: float
35     discount: float
36     insurance: float
37     total: float
38
39
40 class Report(TypedDict):
41     """Підсумок обробки: кількість, сумарний дохід і нові записи."""
42
43     count: int
44     revenue: float
45     rentals: list[Rental]
46
47
48 # Політики, що передаються як функції (typing.Callable)
49 PriceFn = Callable[[float], float] # (price) -> price
50 DiscountFn = Callable[[Rental, float], float] # (rental, base_price) -> discounted
51 InsuranceFn = Callable[[Rental, float], float] # (rental, discounted) -> fee
52 AcceptFn = Callable[[Rental], bool] # (rental) -> чи брати до розрахунку
53 NowFn = Callable[[], float] # () -> поточний час (інжекція залежності)
54 Processor = Callable[[Iterable[Rental], Report]]
55
56
57 # -----
58 # Базові чисті функції предметної області
59 # -----
```

```
62 def base_price(rental: Rental) -> float:
63     """Базова вартість прокату: кількість днів x денна ставка."""
64     return rental["days"] * rental["daily_rate"]
65
66
67 def is_confirmed(rental: Rental) -> bool:
68     """Чи підтверджено прокат (аналог ознаки «оплачено»)."""
69     return rental["status"] == "confirmed"
70
71
72 def duration_discount_rate(days: int, tiers: Sequence[tuple[int, float]]) -> float:
73     """Ставка знижки за тривалість.
74
75     ``tiers`` - послідовність пар ``(мінімальна_кількість_днів, ставка)``.
76     Обирається найбільша ставка серед порогів, які досягнуто.
77     Якщо жоден поріг не досягнуто - 0.0.
78     """
79     reached = [rate for min_days, rate in tiers if days >= min_days]
80     return max(reached, default=0.0)
81
82
83 def insurance_rate_for(
84     car_class: str, rates: Mapping[str, float], default: float
85 ) -> float:
86     """Ставка страхового збору для класу авто (або ``default``)."""
87     return rates.get(car_class, default)
88
89
90 def with_pricing(
91     rental: Rental, *, base: float, discount: float, insurance: float
92 ) -> Rental:
93     """Повертає НОВИЙ запис із доданими полями ціни. Вхідний не чіпаємо."""
94     return {
95         **rental,
96         "base_price": base,
97         "discount": discount,
98         "insurance": insurance,
99         "total": base - discount + insurance,
100     }
101
102
103 def summarize(rentals: Sequence[Rental]) -> Report:
104     """Збирає звіт з уже оцінених записів (без мутацій, без I/O)."""
105     return {
106         "count": len(rentals),
107         "revenue": sum(r["total"] for r in rentals),
108         "rentals": list(rentals),
109     }
110
111
112 # -----
113 # Завдання 1: переписаний process_rentals - усі параметри явні
114 # -----
```

```
117 def process_rentals_pure(  
118     rentals: Iterable[Rental],  
119     *,  
120     min_days: int,  
121     discount_tiers: Sequence[tuple[int, float]],  
122     insurance_rates: Mapping[str, float],  
123     default_insurance_rate: float,  
124 ) -> Report:  
125     """Відбирає підтверджені прокати та обчислює ціну, знижку і страховку.  
126  
127     Жодних глобалів: пороги знижки та ставки страхування приходять аргументами.  
128     Жодних мутацій: кожен запис у результаті - новий словник.  
129     """  
130     confirmed = (r for r in rentals if is_confirmed(r))  
131     qualified = (r for r in confirmed if r["days"] >= min_days)  
132  
133     priced: list[Rental] = []  
134     for r in qualified:  
135         base = base_price(r)  
136         discount = base * duration_discount_rate(r["days"], discount_tiers)  
137         discounted = base - discount  
138         insurance = discounted * insurance_rate_for(  
139             r["car_class"], insurance_rates, default_insurance_rate  
140         )  
141         priced.append(  
142             with_pricing(r, base=base, discount=discount, insurance=insurance)  
143         )  
144  
145     return summarize(priced)  
146  
147  
148 # -----  
149 # Завдання 2: параметризація через typing.Callable (фабрика обробника)  
150 # -----  
151  
152  
153 def make_processor(  
154     *,  
155     accept: AcceptFn,  
156     apply_discount: DiscountFn,  
157     insurance_fee: InsuranceFn,  
158 ) -> Processor:  
159     """Фабрика: повертає налаштований обробник прокатів.  
160  
161     Усі політики (фільтр, знижка, страхування) - функції, тому будь-яку  
162     можна замінити, не змінюючи код обробника (компонувальний підхід).  
163     """  
164  
165     def price_one(rental: Rental) -> Rental:  
166         base = base_price(rental)  
167         discounted = apply_discount(rental, base)  
168         fee = insurance_fee(rental, discounted)  
169         return with_pricing(  
170             rental, base=base, discount=base - discounted, insurance=fee  
171         )  
172  
173     def process(rentals: Iterable[Rental]) -> Report:  
174         priced = [price_one(r) for r in rentals if accept(r)]  
175         return summarize(priced)  
176  
177     return process
```

```
179 # ---- Фабрики політик (функції вищого порядку) -----
180
181
182 def make_min_days_filter(min_days: int) -> AcceptFn:
183     """Приймати лише підтверджені прокати тривалістю не менше ``min_days``."""
184     return lambda r: is_confirmed(r) and r["days"] >= min_days
185
186
187 def make_duration_discount(tiers: Sequence[tuple[int, float]]) -> DiscountFn:
188     """Знижка за тривалість: повертає ціну ПІСЛЯ знижки."""
189     frozen = tuple(tiers) # власна копія - зовнішній список не впливає
190     return lambda r, price: price * (1 - duration_discount_rate(r["days"], frozen))
191
192
193 def make_class_insurance(
194     rates: Mapping[str, float], default: float = 0.0
195 ) -> InsuranceFn:
196     """Страховий збір як відсоток від ціни залежно від класу авто."""
197     frozen = dict(rates)
198     return lambda r, price: price * insurance_rate_for(r["car_class"], frozen, default)
199
200
201 def make_per_day_insurance(fee_per_day: float) -> InsuranceFn:
202     """Альтернативна політика: фіксований страховий збір за кожен день."""
203     return lambda r, _price: r["days"] * fee_per_day
204
205
206 def lift_price_fn(fn: PriceFn) -> DiscountFn:
207     """Перетворює просту функцію ``price -> price`` на політику знижки."""
208     return lambda _r, price: fn(price)
209
210
211 # -----
212 # Завдання 3: інжекція залежностей (час) без втрати чистоти
213 # -----
214
215
216 def stamp_total(total: float, now: NowFn) -> tuple[float, float]:
217     """Чиста щодо ``total``: «час» приходиться ззовні через ``now``."""
218     return total, now()
219
220
221 def stamp_report(report: Report, now: NowFn) -> dict[str, object]:
222     """Повертає новий словник звіту з позначкою часу (звіт не змінюється)."""
223     return {**report, "generated_at": now()}
224
225
226 # -----
227 # Додаткові вправи: compose, make_multiplier, без мутацій списків
228 # -----
229
230 A = TypeVar("A")
231 B = TypeVar("B")
232 C = TypeVar("C")
233
234
235 def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
236     """compose(f, g)(x) == f(g(x))."""
237     return lambda x: f(g(x))
```

```
241 def pipe(*fns: PriceFn) -> PriceFn:
242     """pipe(f, g, h)(x) == h(g(f(x))) - зліва направо, зручно для знижок."""
243
244     def identity(x: float) -> float:
245         return x
246
247     result: PriceFn = identity
248     for fn in fns:
249         result = compose(fn, result)
250     return result
251
252
253 def make_multiplier(k: int) -> Callable[[int], int]:
254     """Фабрика множників: make_multiplier(3)(5) == 15."""
255     return lambda x: x * k
256
257
258 def make_percent_off(percent: float) -> PriceFn:
259     """Знижка у відсотках як функція ціни."""
260     return lambda price: price * (1 - percent / 100)
261
262
263 def make_coupon(amount: float) -> PriceFn:
264     """Купон на фіксовану суму (ціна не стає від'ємною)."""
265     return lambda price: max(0.0, price - amount)
266
267
268 def mark_cancelled(rentals: Sequence[Rental], ids: Iterable[int]) -> list[Rental]:
269     """Замість зміни списку «на місці» повертає новий список нових записів."""
270     to_cancel = frozenset(ids)
271     return [
272         {**r, "status": "cancelled"} if r["id"] in to_cancel else r for r in rentals
273     ]
274
275
276 def totals_by_class(rentals: Sequence[Rental]) -> dict[str, float]:
277     """Сума total за класами авто - новий словник, без накопичувальних мутацій."""
278     classes = sorted({r["car_class"] for r in rentals})
279     return {
280         cls: sum(r["total"] for r in rentals if r["car_class"] == cls)
281         for cls in classes
282     }
```

Файл app.py

app.py

рядки 1-60

```
1  """Оболонка побічних ефектів (I/O shell) для варіанта 9 - прокат автомобілів.
2
3  Тут і ТІЛЬКИ тут живуть побічні ефекти: читання файлу, print, time.time.
4  Ядро (core.py) лише отримує дані й повертає новий результат.
5
6  Запуск:
7      python app.py                # дані з data/rentals.json
8      python app.py шлях/до/файлу  # інший JSON-файл
9  """
10
11 from __future__ import annotations
12
13 import json
14 import sys
15 import time
16 from collections.abc import Sequence
17 from datetime import datetime, timezone
18 from pathlib import Path
19
20 from core import (
21     Rental,
22     Report,
23     lift_price_fn,
24     make_class_insurance,
25     make_coupon,
26     make_duration_discount,
27     make_min_days_filter,
28     make_per_day_insurance,
29     make_percent_off,
30     make_processor,
31     pipe,
32     process_rentals_pure,
33     stamp_report,
34     totals_by_class,
35 )
36
37 DATA_FILE = Path(__file__).with_name("data") / "rentals.json"
38
39 # Конфігурація політик - звичайні дані, які передаються в ядро явно.
40 DISCOUNT_TIERS: Sequence[tuple[int, float]] = ((7, 0.10), (14, 0.15), (30, 0.25))
41 INSURANCE_RATES = {
42     "economy": 0.05,
43     "comfort": 0.08,
44     "business": 0.12,
45     "premium": 0.18,
46 }
47
48
49 # -----
50 # I/O: читання
51 # -----
52
53
54 def load_rentals(path: Path) -> list[Rental]:
55     """Читає список прокатів із JSON-файлу (побічний ефект - файл)."""
56     with path.open(encoding="utf-8") as fh:
57         data: list[Rental] = json.load(fh)
58     return data
```

```
61 # -----
62 # I/O: рендеринг звіту (лише споживає чистий результат)
63 # -----
64
65
66 def render_report(title: str, report: Report) -> None:
67     """Друкує звіт. Нічого не обчислює і нічого не змінює."""
68     print(f"=== {title} ===")
69     print(
70         f"{'ID':>4} {'Клас':<10} {'Днів':>4} {'База':>10} {'Знижка':>9} "
71         f"{'Страх.':>9} {'Разом':>10}"
72     )
73     for r in report["rentals"]:
74         print(
75             f"{r['id']:>4} {r['car_class']:<10} {r['days']:>4} "
76             f"{r['base_price']:>10.2f} {r['discount']:>9.2f} "
77             f"{r['insurance']:>9.2f} {r['total']:>10.2f}"
78         )
79     print(f"Прийнято прокатів: {report['count']}")
80     print(f"Дохід: {report['revenue']:.2f}")
81     by_class = totals_by_class(report["rentals"])
82     for cls, total in by_class.items():
83         print(f" {cls:<10} {total:>10.2f}")
84     print()
85
86
87 def render_stamped(report: Report) -> None:
88     """Демонстрація інжекції часу: time.time передається в ядро як Callable."""
89     stamped = stamp_report(report, time.time)
90     ts = float(str(stamped["generated_at"]))
91     when = datetime.fromtimestamp(ts, tz=timezone.utc).isoformat(timespec="seconds")
92     print(f"Звіт сформовано: {when} (UTC)")
93     print()
94
95
96 # -----
97 # Точка входу
98 # -----
99
100
101 def run(rentals: Sequence[Rental]) -> None:
102     # Завдання 1: чиста функція з явними параметрами (без глобалів).
103     basic = process_rentals_pure(
104         rentals,
105         min_days=2,
106         discount_tiers=DISCOUNT_TIERS,
107         insurance_rates=INSURANCE_RATES,
108         default_insurance_rate=0.10,
109     )
110     render_report("Завдання 1: process_rentals_pure", basic)
111
112     # Завдання 2: той самий розрахунок через фабрику з Callable-політиками.
113     standard = make_processor(
114         accept=make_min_days_filter(2),
115         apply_discount=make_duration_discount(DISCOUNT_TIERS),
116         insurance_fee=make_class_insurance(INSURANCE_RATES, default=0.10),
117     )
118     render_report("Завдання 2: make_processor (стандартні політики)", standard(rentals))
```

app.py

рядки 120-146

```
120     # Інша конфігурація: купон + відсоткова знижка (compose/pipe),
121     # фіксована страховка за день, без обмеження на тривалість.
122     promo_discount = pipe(make_percent_off(5), make_coupon(50.0))
123     promo = make_processor(
124         accept=make_min_days_filter(1),
125         apply_discount=lift_price_fn(promo_discount),
126         insurance_fee=make_per_day_insurance(15.0),
127     )
128     render_report("Завдання 2: make_processor (акційні політики)", promo(rentals))
129
130     # Завдання 3: час приходить ззовні, ядро лишається чистим.
131     render_stamped(basic)
132
133
134 def main(argv: Sequence[str]) -> int:
135     path = Path(argv[1]) if len(argv) > 1 else DATA_FILE
136     try:
137         rentals = load_rentals(path)
138     except (OSError, json.JSONDecodeError) as exc:
139         print(f"Не вдалося прочитати {path}: {exc}", file=sys.stderr)
140         return 1
141     run(rentals)
142     return 0
143
144
145 if __name__ == "__main__":
146     raise SystemExit(main(sys.argv))
```

Файл tests/test_core.py

```
test_core.py      рядки 1-62

1  """pytest-тести чистого ядра (варіант 9 - прокат автомобілів)."""
2
3  from __future__ import annotations
4
5  from collections.abc import Callable
6  from copy import deepcopy
7
8  import pytest
9
10 from core import (
11     DiscountFn,
12     InsuranceFn,
13     Rental,
14     Report,
15     base_price,
16     compose,
17     duration_discount_rate,
18     insurance_rate_for,
19     lift_price_fn,
20     make_class_insurance,
21     make_coupon,
22     make_duration_discount,
23     make_min_days_filter,
24     make_multiplier,
25     make_per_day_insurance,
26     make_percent_off,
27     make_processor,
28     mark_cancelled,
29     pipe,
30     process_rentals_pure,
31     stamp_report,
32     stamp_total,
33     totals_by_class,
34     with_pricing,
35 )
36
37 TIERS = ((7, 0.10), (30, 0.25))
38 RATES = {"economy": 0.05, "comfort": 0.08, "business": 0.12, "premium": 0.18}
39
40
41 def run_pure(rentals: list[Rental]) -> Report:
42     """Стандартна конфігурація process_rentals_pure для тестів."""
43     return process_rentals_pure(
44         rentals,
45         min_days=2,
46         discount_tiers=TIERS,
47         insurance_rates=RATES,
48         default_insurance_rate=0.10,
49     )
50
51
52 # -----
53 # Референтна прозорість та відсутність мутацій
54 # -----
55
56
57 def test_referential_transparency(sample_rentals: list[Rental]) -> None:
58     r1 = run_pure(sample_rentals)
59     r2 = run_pure(sample_rentals)
60     assert r1 == r2 # той самий вхід -> той самий вихід
```

```
63 def test_no_mutation(sample_rentals: list[Rental]) -> None:
64     original = deepcopy(sample_rentals)
65     run_pure(sample_rentals)
66     assert sample_rentals == original # вхід не змінюється
67
68
69 def test_result_records_are_new_objects(sample_rentals: list[Rental]) -> None:
70     result = run_pure(sample_rentals)
71     inputs = {id(r) for r in sample_rentals}
72     assert all(id(r) not in inputs for r in result["rentals"])
73     assert all("total" not in r for r in sample_rentals)
74
75
76 def test_processor_no_mutation(sample_rentals: list[Rental]) -> None:
77     original = deepcopy(sample_rentals)
78     processor = make_processor(
79         accept=make_min_days_filter(0),
80         apply_discount=make_duration_discount(TIERS),
81         insurance_fee=make_class_insurance(RATES, default=0.1),
82     )
83     processor(sample_rentals)
84     assert sample_rentals == original
85
86
87 # -----
88 # Коректність обчислень
89 # -----
90
91
92 def test_base_price() -> None:
93     rental: Rental = {
94         "id": 1,
95         "car_class": "economy",
96         "days": 3,
97         "daily_rate": 100.0,
98         "status": "confirmed",
99     }
100     assert base_price(rental) == 300.0
101
102
103 def test_duration_discount_rate() -> None:
104     assert duration_discount_rate(3, TIERS) == 0.0
105     assert duration_discount_rate(7, TIERS) == 0.10
106     assert duration_discount_rate(29, TIERS) == 0.10
107     assert duration_discount_rate(30, TIERS) == 0.25
108     assert duration_discount_rate(100, ()) == 0.0
109
110
111 def test_insurance_rate_for() -> None:
112     assert insurance_rate_for("premium", RATES, 0.1) == 0.18
113     assert insurance_rate_for("unknown", RATES, 0.1) == 0.1
```

```
116 def test_with_pricing_returns_new_dict() -> None:
117     rental: Rental = {
118         "id": 1,
119         "car_class": "economy",
120         "days": 3,
121         "daily_rate": 100.0,
122         "status": "confirmed",
123     }
124     priced = with_pricing(rental, base=300.0, discount=30.0, insurance=13.5)
125     assert priced is not rental
126     assert "total" not in rental
127     assert priced["total"] == pytest.approx(283.5)
128     assert priced["id"] == 1
129
130
131 def test_process_rentals_pure_values(sample_rentals: list[Rental]) -> None:
132     result = run_pure(sample_rentals)
133     # Відкинуто: id=3 (1 день < 2), id=5 (cancelled), id=6 (pending)
134     assert [r["id"] for r in result["rentals"]] == [1, 2, 4]
135     assert result["count"] == 3
136
137     by_id = {r["id"]: r for r in result["rentals"]}
138     # id=1: 3*100 = 300, знижка 0, страх. 5% -> 315
139     assert by_id[1]["total"] == pytest.approx(315.0)
140     # id=2: 7*200 = 1400, знижка 10% = 140 -> 1260, страх. 8% = 100.8 -> 1360.8
141     assert by_id[2]["discount"] == pytest.approx(140.0)
142     assert by_id[2]["total"] == pytest.approx(1360.8)
143     # id=4: 30*400 = 12000, знижка 25% = 3000 -> 9000, страх. 18% = 1620 -> 10620
144     assert by_id[4]["total"] == pytest.approx(10620.0)
145
146     assert result["revenue"] == pytest.approx(315.0 + 1360.8 + 10620.0)
147
148
149 # -----
150 # Callable-політики
151 # -----
152
153
154 def test_callable_policies(sample_rentals: list[Rental]) -> None:
155     accept = lambda r: r["status"] == "confirmed" and base_price(r) >= 500 # noqa: E731
156     apply_discount: DiscountFn = lambda r, p: p * 0.9 # noqa: E731
157     insurance_fee: InsuranceFn = lambda r, p: p * 0.2 # noqa: E731
158
159     processor = make_processor(
160         accept=accept, apply_discount=apply_discount, insurance_fee=insurance_fee
161     )
162     result = processor(sample_rentals)
163
164     # Пройшли: id=2 (1400), id=3 (500), id=4 (12000); id=1 (300) - відкинуто
165     assert [r["id"] for r in result["rentals"]] == [2, 3, 4]
166     expected = sum(b * 0.9 * 1.2 for b in (1400.0, 500.0, 12000.0))
167     assert result["revenue"] == pytest.approx(expected)
168
169
170 def test_processor_matches_pure_version(sample_rentals: list[Rental]) -> None:
171     processor = make_processor(
172         accept=make_min_days_filter(2),
173         apply_discount=make_duration_discount(TIERS),
174         insurance_fee=make_class_insurance(RATES, default=0.10),
175     )
176     assert processor(sample_rentals) == run_pure(sample_rentals)
```

```
178 def test_swapping_policy_changes_only_that_part(sample_rentals: list[Rental]) -> None:
179     accept = make_min_days_filter(2)
180     discount = make_duration_discount(TIERS)
181     percent = make_processor(
182         accept=accept,
183         apply_discount=discount,
184         insurance_fee=make_class_insurance(RATES, 0.1),
185     )
186     per_day = make_processor(
187         accept=accept,
188         apply_discount=discount,
189         insurance_fee=make_per_day_insurance(10.0),
190     )
191
192     p, d = percent(sample_rentals), per_day(sample_rentals)
193     assert [r["id"] for r in p["rentals"]] == [r["id"] for r in d["rentals"]]
194     for a, b in zip(p["rentals"], d["rentals"], strict=True):
195         assert a["discount"] == pytest.approx(b["discount"])
196         assert b["insurance"] == pytest.approx(b["days"] * 10.0)
197
198
199 def test_make_duration_discount_is_isolated_from_later_changes() -> None:
200     tiers = [(7, 0.5)]
201     policy = make_duration_discount(tiers)
202     tiers.append((1, 0.99)) # зміна зовнішнього списку не впливає на політику
203     rental: Rental = {
204         "id": 1,
205         "car_class": "economy",
206         "days": 2,
207         "daily_rate": 10.0,
208         "status": "confirmed",
209     }
210     assert policy(rental, 20.0) == 20.0
211
212
213 # -----
214 # Інжекція залежностей (час)
215 # -----
216
217
218 def test_stamp_total_uses_injected_clock() -> None:
219     fake_now = lambda: 1_700_000_000.0 # noqa: E731
220     assert stamp_total(99.0, fake_now) == (99.0, 1_700_000_000.0)
221     assert stamp_total(99.0, fake_now) == stamp_total(99.0, fake_now)
222
223
224 def test_stamp_report_does_not_mutate(sample_rentals: list[Rental]) -> None:
225     report = run_pure(sample_rentals)
226     before = deepcopy(report)
227     stamped = stamp_report(report, lambda: 1.0)
228     assert report == before
229     assert stamped["generated_at"] == 1.0
230     assert "generated_at" not in report
231
232
233 # -----
234 # Додаткові вправи
235 # -----
```

```
239 def test_compose() -> None:
240     inc: Callable[[int], int] = lambda x: x + 1 # noqa: E731
241     dbl: Callable[[int], int] = lambda x: x * 2 # noqa: E731
242     assert compose(inc, dbl)(5) == 11 # inc(dbl(5))
243     assert compose(dbl, inc)(5) == 12 # dbl(inc(5))
244
245
246 def test_pipe_builds_discount_from_small_functions() -> None:
247     discount = pipe(make_percent_off(10), make_coupon(20.0))
248     assert discount(200.0) == pytest.approx(160.0) # 200*0.9 = 180, -20 = 160
249     assert make_coupon(500.0)(100.0) == 0.0 # не від'ємна
250
251     rental: Rental = {
252         "id": 1,
253         "car_class": "economy",
254         "days": 1,
255         "daily_rate": 200.0,
256         "status": "confirmed",
257     }
258     assert lift_price_fn(discount)(rental, 200.0) == pytest.approx(160.0)
259
260
261 def test_make_multiplier() -> None:
262     triple = make_multiplier(3)
263     assert triple(5) == 15
264     assert [make_multiplier(k)(2) for k in (1, 2, 3)] == [2, 4, 6]
265
266
267 def test_mark_cancelled_returns_new_list(sample_rentals: list[Rental]) -> None:
268     original = deepcopy(sample_rentals)
269     updated = mark_cancelled(sample_rentals, [1, 2])
270     assert sample_rentals == original
271     assert updated is not sample_rentals
272     assert [r["status"] for r in updated[:2]] == ["cancelled", "cancelled"]
273     assert updated[2] is sample_rentals[2] # незміннені записи можна ділити
274
275
276 def test_totals_by_class(sample_rentals: list[Rental]) -> None:
277     result = run_pure(sample_rentals)
278     totals = totals_by_class(result["rentals"])
279     assert set(totals) == {"economy", "comfort", "premium"}
280     assert sum(totals.values()) == pytest.approx(result["revenue"])
```

Запуск програми

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1> py app.py
=== Завдання 1: process_rentals_pure ===
  ID Клас      Днів      База      Знижка      Страх.      Разом
  ---
  1 economy    3         2400.00    0.00       120.00      2520.00
  2 comfort    7         8400.00    840.00     604.80      8164.80
  4 premium    14        56000.00   8400.00    8568.00     56168.00
  6 comfort    30        33000.00   8250.00    1980.00     26730.00
  8 suv        4         7600.00    0.00       760.00      8360.00
Прийнято прокатів: 5
Дохід:          101942.80
  comfort      34894.80
  economy      2520.00
  premium      56168.00
  suv          8360.00

=== Завдання 2: make_processor (стандартні політики) ===
  ID Клас      Днів      База      Знижка      Страх.      Разом
  ---
  1 economy    3         2400.00    0.00       120.00      2520.00
  2 comfort    7         8400.00    840.00     604.80      8164.80
  4 premium    14        56000.00   8400.00    8568.00     56168.00
  6 comfort    30        33000.00   8250.00    1980.00     26730.00
  8 suv        4         7600.00    0.00       760.00      8360.00
Прийнято прокатів: 5
Дохід:          101942.80
  comfort      34894.80
  economy      2520.00
  premium      56168.00
  suv          8360.00

=== Завдання 2: make_processor (акційні політики) ===
  ID Клас      Днів      База      Знижка      Страх.      Разом
  ---
  1 economy    3         2400.00    170.00     45.00      2275.00
  2 comfort    7         8400.00    470.00    105.00     8035.00
  3 business    1         2500.00    175.00     15.00      2340.00
  4 premium    14        56000.00   2850.00    210.00     53360.00
  6 comfort    30        33000.00   1700.00    450.00     31750.00
  8 suv        4         7600.00    430.00     60.00      7230.00
Прийнято прокатів: 6
Дохід:          104990.00
  business      2340.00
  comfort      39785.00
  economy      2275.00
  premium      53360.00
  suv          7230.00

Звіт сформовано: 2026-10-09T07:28:15+00:00 (UTC)
```

Рис. 1. Вивід програми app.py: звіти за двома наборами політик і позначка часу

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1> py -m pytest -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 20 items

tests/test_core.py::test_referential_transparency PASSED [ 5%]
tests/test_core.py::test_no_mutation PASSED [ 10%]
tests/test_core.py::test_result_records_are_new_objects PASSED [ 15%]
tests/test_core.py::test_processor_no_mutation PASSED [ 20%]
tests/test_core.py::test_base_price PASSED [ 25%]
tests/test_core.py::test_duration_discount_rate PASSED [ 30%]
tests/test_core.py::test_insurance_rate_for PASSED [ 35%]
tests/test_core.py::test_with_pricing_returns_new_dict PASSED [ 40%]
tests/test_core.py::test_process_rentals_pure_values PASSED [ 45%]
tests/test_core.py::test_callable_policies PASSED [ 50%]
tests/test_core.py::test_processor_matches_pure_version PASSED [ 55%]
tests/test_core.py::test_swapping_policy_changes_only_that_part PASSED [ 60%]
tests/test_core.py::test_make_duration_discount_is_isolated_from_later_changes PASSED [ 65%]
tests/test_core.py::test_stamp_total_uses_injected_clock PASSED [ 70%]
tests/test_core.py::test_stamp_report_does_not_mutate PASSED [ 75%]
tests/test_core.py::test_compose PASSED [ 80%]
tests/test_core.py::test_pipe_builds_discount_from_small_functions PASSED [ 85%]
tests/test_core.py::test_make_multiplier PASSED [ 90%]
tests/test_core.py::test_mark_cancelled_returns_new_list PASSED [ 95%]
tests/test_core.py::test_totals_by_class PASSED [100%]

===== 20 passed in 0.10s =====
```

Рис. 2. Результат запуску pytest: 20 тестів пройдено

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1> py -m mypy
Success: no issues found in 4 source files
PS D:\IPZ2kurs\func\lab1> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1> py -m black --check .
All done!
5 files would be left unchanged.
```

Рис. 3. mypy (strict), ruff та black без зауважень

Висновок:

Під час виконання лабораторної роботи початковий імперативний код на Python було успішно переписано у функціональному стилі. На практиці опрацьовано створення чистих функцій предметної області та відокремлення обчислень від введення-виведення: усі `print`, читання файлів і звернення до системного часу зосереджено в оболонці `app.py`, а ядро `core.py` не має побічних ефектів. Завдяки `typing.Callable` політики знижки, страхування та фільтрації передаються як звичайні аргументи, а фабрика `make_processor` дозволяє збирати обробник із будь-якого їх набору. Розроблений код не використовує глобальних змінних і не змінює вхідних даних, що робить його референтно прозорим, передбачуваним і легким для тестування, що підтверджено 20 `pytest`-тестами та перевіркою туру.